EWB PYTHON BACKEND DEVELOPMENT INTERNSHIP

Project and assignment submission

Student Records API

FastAPI / SQLite / Optional Ollama extension

A complete student database backend with validated CRUD operations, persistent storage, automated tests and a reproducible API demonstration.

Student name

Student / intern ID

College / institution

Submission date

Prepared for submission to EWB Edu Tech Pvt. Ltd.

Version 1.0.0 | Verification date: 18 October 2026

01 / Project overview

Abstract

The Student Records API is a Python backend that manages student profiles through Create, Read, Update and Delete operations. FastAPI exposes the HTTP interface, Pydantic validates input, and SQLite stores records in a persistent file. The project includes interactive API documentation, automated tests, sample requests and a reproducible demonstration.

The application addresses a simple administrative need: store consistent student details, retrieve specific profiles, find matching students and safely maintain the information over time. It uses fictional records for demonstration. A separate optional extension retrieves database facts for natural-language questions and can request local Ollama commentary.

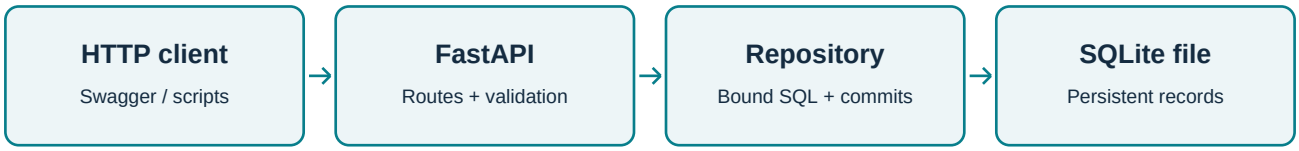
Objectives and delivered requirements

Requirement	Implementation / evidence
Python + FastAPI + SQLite	Organized app package, pinned dependencies and a file-backed database.
All four CRUD operations	POST, GET, PUT, PATCH and DELETE routes; live HTTP response transcript.
Student profile fields	Generated ID, name, date of birth, email and/or phone; optional course, address and enrollment date.
Read and filter profiles	ID lookup, name/course/email/date filters and paginated collections.
Validation and missing records	422 input errors, 404 missing-record errors and 409 duplicate-email errors.
Persistent, accurate storage	Application-restart tests and live checks across four Uvicorn processes.
Reproducible submission	README, requirements, tests, HTTP requests, Postman collection and this report.

Project scope

This is a local backend demonstration for the EWB Python Backend Development Internship. Swagger UI supplies the interaction surface; a separate frontend is not required. The core application does not require Ollama, a cloud account or an API key.

02 / Architecture and database



The route layer handles HTTP requests and responses. Schemas validate field values and complete-record rules. The repository owns all SQL and transaction boundaries. Configuration is read when the application is created, which also allows tests to use isolated databases.

Student table design

Field	Storage	Rule / purpose
student_id	INTEGER	Primary key; generated with AUTOINCREMENT.
name	TEXT	Required; trimmed length 2-100.
date_of_birth	TEXT	Required ISO date; not in the future.
email	TEXT / NULL	Syntax-validated; unique and case-insensitive.
phone	TEXT / NULL	7-15 digits with permitted separators.
course	TEXT / NULL	Up to 100 characters; indexed.
address	TEXT / NULL	Up to 300 characters.
enrollment_date	TEXT	Defaults to today; not earlier than birth date.
created_at	TEXT	UTC timestamp assigned at creation.
updated_at	TEXT	UTC timestamp refreshed on update.

Persistence and consistency

The default file is data/students.db under the project folder. Startup creates the schema if needed and enables SQLite WAL mode. Each operation opens and closes its own connection. Successful writes commit; failed writes roll back. Email uniqueness and contact/date checks are also enforced by the database.

PATCH acquires a write reservation with BEGIN IMMEDIATE before reading and merging the old record. This prevents two updates to different fields from losing one another's changes. List count and page retrieval share one transaction. Indexes support course and birth-date filtering.

03 / API design and CRUD flow

Method	Endpoint	Success / action
POST	/students	201 - create; returns the saved record.
GET	/students	200 - filtered and paginated collection.
GET	/students/{student_id}	200 - retrieve an individual profile.
PUT	/students/{student_id}	200 - replace all editable fields.
PATCH	/students/{student_id}	200 - update only supplied fields.
DELETE	/students/{student_id}	204 - remove; empty response body.
GET	/health	200 - application/database check.
POST	/ask	200 - optional read-only assistant.

Example create payload

```
{
  "name": "Asha Demo",
  "date_of_birth": "2006-04-12",
  "email": "asha.demo@example.com",
  "course": "Python",
  "enrollment_date": "2026-01-10"
}
```

The server adds the student ID, timestamps and omitted optional fields. A Location response header identifies the new resource. IDs are immutable and cannot be supplied in a request body.

Operation algorithms

Create: validate the request, bind values to an INSERT, commit and return the new row. Read: validate query parameters, bind filter values, count matches and return an ordered slice. Update: locate the record, replace or merge its fields, validate the resulting record and commit. Delete: remove the row by its ID and check that one row was affected.

PUT requires the creation schema and resets omitted optional fields to defaults. PATCH preserves omitted fields. Explicit null can clear an optional field only when all record rules still hold. Missing IDs never create new records implicitly.

04 / Validation and safe queries

Validation rules and failure behavior

Condition	Behavior
Missing / blank / oversized name	422; names must have 2-100 trimmed characters.
Invalid or future birth date	422; calendar dates are validated.
Neither email nor phone provided	422; at least one valid contact is required.
Malformed email or phone	422; syntax and phone digit limits are enforced.
Email already exists	409; case-insensitive uniqueness check.
Enrollment before birth date	422; merged records are checked after PATCH.
Missing student ID	404; no record is created by an update.
Unknown fields or empty PATCH	422; accidental field names and empty updates are rejected.
Invalid pagination or ID bounds	422; numeric values fit supported ranges.
Database / AI unavailable	503; response avoids internal paths and service details.

Filtering semantics

Name filtering is a literal substring search; percent signs and underscores are not wildcard operators. Course and email are exact matches. Case-insensitivity uses SQLite's built-in ASCII-oriented rules. Multiple filters combine with AND. Birth-date boundaries are exclusive: born_after=2005-12-31 selects dates in 2006 or later.

Collections return total, limit, offset and items. The default limit is 20, the maximum is 100, and offset is non-negative. Stable ascending student-ID ordering makes pagination predictable. An empty result is a valid 200 response with an empty items list.

SQL and data handling

All user values are passed using SQLite placeholders. Only fixed application-owned SQL fragments are assembled. Failed writes roll back. Fictional example records use example.com email addresses; the ZIP excludes database files, virtual environments and caches. The optional model cannot execute queries.

05 / Verification and results

Verification was completed on 18 October 2026 using Python 3.14.6 on Windows. The final automated run passed 48 tests in 5.41 seconds. A separate live verifier exercised HTTP requests and checked persistence across four independent Uvicorn server processes.

Verification area	Observed result
CRUD lifecycle	Create, read, PATCH, PUT and delete passed.
Input validation	Invalid names, contacts, dates and unknown fields were rejected without writes.
Duplicates and rollback	Case-insensitive duplicate email returned 409; rejected updates preserved old values.
Filtering and pagination	Combined filters, date boundaries, total count and empty pages passed.
SQL injection attempts	Injection-like strings and wildcard characters were treated as literal data.
Concurrency	Parallel partial updates preserved each changed field.
Persistence	Create, update and delete each survived an application/server restart.
Documentation and service	Health check, Swagger endpoint and OpenAPI routes passed.
Optional assistant	Count/filter/bounds/privacy checks passed; mocked Ollama success and failure paths passed.

Recorded live HTTP sequence

```
GET    /health                       -> 200
POST   /students                     -> 201
GET    /students/1                   -> 200
GET    /students?course=Python&limit=2 -> 200
PATCH  /students/1                   -> 200
PUT    /students/1                   -> 200
POST   /students (duplicate)         -> 409
POST   /students (invalid date)      -> 422
POST   /ask                          -> 200
DELETE /students/1                   -> 204
GET    /students/1                   -> 404
```

Evidence: docs/test-results.txt, docs/live-demo.json and docs/live-verification.json. The test client emitted one dependency deprecation warning; all tests passed. Ollama was mocked for contract tests; an actual local model was not run. These results do not claim production load testing.

06 / Installation and demonstration

Windows quick start

Extract the project ZIP and open PowerShell inside the ewb-student-crud folder. Install Python first if necessary. Run:

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

Open http://127.0.0.1:8000/docs. The server binds to loopback by default. If the py launcher is unavailable, use python in the first command. The README includes macOS/Linux equivalents. Activation is unnecessary when using the virtual environment's Python executable directly.

Sample import and live demonstration

```
.\.venv\Scripts\python.exe -m scripts.seed
.\.venv\Scripts\python.exe -m scripts.demo
```

Run these from a second terminal while the API is active. The seed command adds five fictional records and skips existing sample emails. The demo creates a temporary demonstration record, exercises each operation and removes that record when done.

Reproduce verification

```
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m scripts.verify_live
```

The live verifier starts its own loopback server on a temporary port and uses an isolated SQLite database. No pre-existing server is required. It saves actual JSON responses and checks that create, update and delete persist through restarts.

Suggested review sequence

Create one student and note its generated ID. Retrieve it and filter the list by course. PATCH the course and show that the name and email remain unchanged. Demonstrate a duplicate email and an invalid date. Restart the API, retrieve the same record, delete it and show the resulting 404. Finally, show the tests and explain the source modules.

Swagger's UI assets load from a CDN. For offline API demonstrations, use scripts/demo.py, the HTTP requests or the exported OpenAPI JSON. Changing STUDENT_DB_PATH selects a different database file.

07 / Optional local AI extension

Retrieval before generation

The /ask route recognizes a small set of documented natural-language patterns. Backend code retrieves relevant records with parameterized queries and constructs an accurate answer from those results. With use_ai=true, retrieved facts are also sent to a local Ollama model to generate supplementary commentary.

Question	Backend action
How many students are there?	Count all student rows.
List students	Return up to 50 students and the full total.
List students born after 2005	Filter date_of_birth after 2005-12-31.
List students in course Python	Filter by exact, case-insensitive course.

Local setup

```
ollama pull llama3.2:1b
$env:OLLAMA_ENABLED = "1"
$env:OLLAMA_MODEL = "llama3.2:1b"
```

Start Ollama locally, then start or restart the API after setting these environment variables. The API contacts only http://127.0.0.1:11434/api/chat. Set use_ai=false to use the deterministic assistant without a model.

Grounding and boundaries

The prompt includes the exact count, applied filters and a bounded record list. Email, phone and address are excluded. The model has no database connection, SQL execution interface or tools. Unsupported questions receive 422; unavailable or malformed model responses produce 503.

The response keeps answer, total_matches, filters and records as deterministic SQLite evidence. The separate ai_explanation field is generated text and may contain mistakes. If more than 50 records match, shown and truncated make the limit explicit while total_matches retains the full count.

Verification limit

The HTTP contract, prompt contents and error handling were tested with mocks. A local model was not downloaded or executed during verification. The core assignment remains fully usable without this optional feature.

08 / Review notes and conclusion

Key viva answers

Question	Answer
Why FastAPI?	Typed request/response handling, validation integration and generated API documentation.
Why SQLite?	Persistent relational storage without a separate database service; suitable for a small local demo.
PUT or PATCH?	PUT replaces editable fields; PATCH merges only supplied fields and validates the complete result.
How is SQL injection avoided?	Values use bound placeholders; SQL structure is application-owned.
What proves persistence?	Saved create/update/delete results survive separate server process restarts.
What would production need?	Authentication, authorization, auditing, backups, migrations and operational monitoring.

Conclusion and future work

The project fulfills the required student CRUD workflow with persistent storage, input validation, predictable errors and reproducible documentation. Automated and real HTTP checks support the implementation. The optional assistant demonstrates controlled retrieval while keeping database facts separate from generated text.

The current scope is a local internship demonstration. It does not include authentication, role management, production deployment or large-scale load testing. Future work can add these capabilities, managed schema migrations and a server database when required by deployment size.

Submission checklist

Fill the blank cover fields. Run the demonstration and review the code explanations in docs/VIVA_AND_DEMO.md. Upload the project contents to a GitHub repository, preserving subfolders and excluding generated databases or environments. Verify trainer access if the repository is private, then submit its link through the announced trainer channel.

References

1. EWB Edu Tech Pvt. Ltd., Internship Project Assignment: Students Database CRUD Application, supplied brief.
2. FastAPI testing documentation: https://fastapi.tiangolo.com/tutorial/testing/
3. Python sqlite3 documentation: https://docs.python.org/3/library/sqlite3.html
4. Pydantic documentation: https://docs.pydantic.dev/latest/
5. Ollama chat API: https://docs.ollama.com/api/chat
Documentation consulted on 18 October 2026. Additional reproducible examples appear in the project README.